Architecture and concepts

eBPF maps

eBPF maps are efficient key/value stores that reside in kernel space and can be used to share data among multiple eBPF programs or to communicate between a user space application and eBPF code running in the kernel. eBPF programs can leverage eBPF maps to store and retrieve data in a wide set of data structures. Map implementations are provided by the core kernel. There are various types,44(https://en.wikipedia.org/wiki/EBPF#cite_note-44) including hash maps, arrays, and ring buffers.

In practice, eBPF maps are typically used for scenarios such as a user space program writing configuration information to be retrieved by an eBPF program, an eBPF program storing state for later retrieval by another eBPF program (or a future run of the same program), or an eBPF program writing results or metrics into a map for retrieval by a user space program that will present results.45(https://en.wikipedia.org/wiki/EBPF#cite_note-:0-45)

eBPF virtual machine

The eBPF virtual machine runs within the kernel and takes in a program in the form of eBPF bytecode instructions which are converted to native machine instructions that run on the CPU. Early implementations of eBPF saw eBPF bytecode interpreted, but this has now been replaced with a Just-in-Time (JIT) compilation process for performance and security-related reasons.45(https://en.wikipedia.org/wiki/EBPF#cite_note-:0-45)

The eBPF virtual machine consists of eleven 64-bit registers with 32-bit subregisters, a program counter and a 512-byte large BPF stack space. These general purpose registers keep track of state when eBPF programs are executed.46(https://en.wikipedia.org/wiki/EBPF#cite_note-bpf-architecture-cilium-46)

Tail calls

Tail calls can call and execute another eBPF program and replace the execution context, similar to how the execve() system call operates for regular processes. This basically allows an eBPF program to call another eBPF program. Tail calls are implemented as a long jump, reusing the same stack frame. Tail calls are particularly useful in eBPF, where the stack is limited to 512 bytes. During runtime, functionality can be added or replaced atomically, thus altering the BPF program's execution behavior.46(https://en.wikipedia.org/wiki/EBPF#cite_note-bpf-architecture-cilium-46) A popular use case for tail calls is to spread the complexity of eBPF programs over several programs. Another use case is for replacing or extending logic by replacing the contents of the program array while it is in use. For example, to update a program version without downtime or to enable/disable logic.47(https://en.wikipedia.org/wiki/EBPF#cite_note-:10-47)

BPF to BPF calls

It is generally considered good practice in software development to group common code into a function, encapsulating logic for reusability. Prior to Linux kernel 4.16 and LLVM 6.0, a typical eBPF C program had to explicitly direct the compiler to inline a function, resulting in a BPF object file that had duplicate functions. This restriction was lifted, and mainstream eBPF compilers now support writing functions naturally in eBPF programs. This reduces the generated eBPF code size, making it friendlier to a CPU instruction cache.45(https://en.wikipedia.org/wiki/EBPF#cite_note-:0-45)[[46]](https://en.wikipedia.org/wiki/EBPF#cite_note-bpf-architecture-cilium-46)

eBPF verifier

The verifier is a core component of eBPF, and its main responsibility is to ensure that an eBPF program is safe to execute. It performs a static analysis of the eBPF bytecode to guarantee its safety. The verifier analyzes the program to assess all possible execution paths. It steps through the instructions in order and evaluates them. The verification process starts with a depth-first search through all possible paths of the program, the verifier simulates the execution of each instruction using abstract interpretation,48(https://en.wikipedia.org/wiki/EBPF#cite_note-48) tracking the state of registers and stack if any instruction could lead to an unsafe state, verification fails. This process continues until all paths have been analyzed or a violation is found. Depending on the type of program, the verifier checks for violations of specific rules. These rules can include checking that an eBPF program always terminates within a reasonable amount of time (no infinite loops or infinite recursion), checking that an eBPF program is not allowed to read arbitrary memory because being able to arbitrary read memory could allow a program to leak sensitive information, checking that network programs are not allowed to access memory outside of packet bounds because adjacent memory could contain sensitive information, checking that programs are not allowed to deadlock, so any held spinlocks must be released and only one lock can be held at a time to avoid deadlocks over multiple programs, checking that programs are not allowed to read uninitialized memory. This is not an exhaustive list of the checks the verifier does, and there are exceptions to these rules. An example is that tracing programs have access to helpers that allow them to read memory in a controlled way, but these program types require root privileges and thus do not pose a security risk.47(https://en.wikipedia.org/wiki/EBPF#cite_note-:10-47)[[45]](https://en.wikipedia.org/wiki/EBPF#cite_note-:0-45)

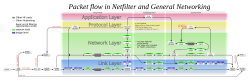
Over time the eBPF verifier has evolved to include newer features and optimizations, such as support for bounded loops, dead-code elimination, function-by-function verification, and callbacks.

eBPF CO-RE (Compile Once - Run Everywhere)

eBPF programs use the memory and data structures from the kernel. Some structures can be modified between different kernel versions, altering the memory layout. Since the Linux kernel is continuously developed, there is no guarantee that the internal data structures will remain the same across different versions. CO-RE is a fundamental concept in modern eBPF development that allows eBPF programs to be portable across different kernel versions and configurations. It addresses the challenge of kernel structure variations between different Linux distributions and versions. CO-RE comprises BTF (BPF Type Format) - a metadata format that describes the types used in the kernel and eBPF programs and provides detailed information about struct layouts, field offsets, and data types. It enables runtime accessibility of kernel types, which is crucial for BPF program development and verification. BTF is included in the kernel image of BTF-enabled kernels. Special relocations are emitted by the compiler (e.g., LLVM). These relocations capture high-level descriptions of what information the eBPF program intends to access. The libbpf library adapts eBPF programs to work with the data structure layout on the target kernel where they run, even if this layout is different from the kernel where the code was compiled. To do this, libbpf needs the BPF CO-RE relocation information generated by Clang as part of the compilation process.45(https://en.wikipedia.org/wiki/EBPF#cite_note-:0-45) The compiled eBPF program is stored in an ELF (Executable and Linkable Format) object file. This file contains BTF-type information and Clang-generated relocations. The ELF format allows the eBPF loader (e.g., libbpf) to process and adjust the BPF program dynamically for the target kernel.49(https://en.wikipedia.org/wiki/EBPF#cite_note-49)

XDP (**eXpress Data Path**) is an eBPF-based high-performance network data path used to send and receive network packets at high rates by bypassing most of the operating system networking stack. It is merged in the Linux kernel since version 4.8.2(https://en.wikipedia.org/wiki/Express_Data_Path#cite_note-2) This implementation is licensed under GPL. Large technology firms including Amazon, Google and Intel support its development. Microsoft released their free and open source implementation *XDP for Windows* in May 2022.1(https://en.wikipedia.org/wiki/Express_Data_Path#cite_note-neowin-2022-05-25-1) It is licensed under MIT License.3(https://en.wikipedia.org/wiki/Express_Data_Path#cite_note-cloud7-2022-05-25-3)

Data path



Packet flow paths in the Linux kernel. XDP bypasses the networking stack and memory allocation for packet metadata.

The idea behind XDP is to add an early hook in the RX path of the kernel, and let a user supplied eBPF program decide the fate of the packet. The hook is placed in the network interface controller (NIC) driver just after the interrupt processing, and before any memory allocation needed by the network stack itself, because memory allocation can be an expensive operation. Due to this design, XDP can drop 26 million packets per second per core with commodity hardware.4(https://en.wikipedia.org/wiki/Express_Data_Path#cite_note-4)

The eBPF program must pass a preverifier test5(https://en.wikipedia.org/wiki/Express_Data_Path#cite_note-5) before being loaded, to avoid executing malicious code in kernel space. The preverifier checks that the program contains no out-of-bounds accesses, loops or global variables.

The program is allowed to edit the packet data and, after the eBPF program returns, an action code determines what to do with the packet:

- `XDP_PASS` : let the packet continue through the network stack
- `XDP_DROP` : silently drop the packet
- `XDP_ABORTED` : drop the packet with trace point exception
- `XDP_TX` : bounce the packet back to the same NIC it arrived on
- `XDP_REDIRECT` : redirect the packet to another NIC or user space socket via the AF_XDP address family

XDP requires support in the NIC driver but, as not all drivers support it, it can fallback to a generic implementation, which performs the eBPF processing in the network stack, though with slower performance.6(https://en.wikipedia.org/wiki/Express_Data_Path#cite_note-6)

XDP has infrastructure to offload the eBPF program to a network interface controller which supports it, reducing the CPU load. In 2023, only Netronome7(https://en.wikipedia.org/wiki/Express_Data_Path#cite_note-7) cards support it.

Microsoft is partnering with other companies and adding support for XDP in its MsQuic implementation of the QUIC protocol.1(https://en.wikipedia.org/wiki/Express_Data_Path#cite_note-neowin-2022-05-25-1)

AF_XDP

Along with XDP, a new address family entered in the Linux kernel starting 4.18.8(https://en.wikipedia.org/wiki/Express_Data_Path#cite_note-8) AF_XDP, formerly known as AF_PACKETv4 (which was never included in the mainline kernel),9(https://en.wikipedia.org/wiki/Express_Data_Path#cite_note-9) is a raw socket optimized for high performance packet processing and allows zero-copy between kernel and applications. As the socket can be used for both receiving and transmitting, it supports high performance network applications purely in user space.10(https://en.wikipedia.org/wiki/Express_Data_Path#cite_note-10)

Graphics code within the OpenGL stack can be left in intermediate representation and then compiled when run on the target machine. On systems with high-end graphics processing units (GPUs), the resulting code remains quite thin, passing the instructions on to the GPU with minimal changes. On systems with low-end GPUs, LLVM will compile optional procedures that run on the local central processing unit (CPU) that emulate instructions that the GPU cannot run internally. LLVM improved performance on low-end machines using Intel GMA chipsets. A similar system was developed under the Gallium3D LLVMpipe, and incorporated into the GNOME shell to allow it to run without a proper 3D hardware driver loaded.41(https://en.wikipedia.org/wiki/LLVM#cite_note-41)

In 2011, programs compiled by GCC outperformed those from LLVM by 10%, on average.42(https://en.wikipedia.org/wiki/LLVM#cite_note-42)[[43]](https://en.wikipedia.org/wiki/LLVM#cite_note-43) In 2013, phoronix reported that LLVM had caught up with GCC, compiling binaries of approximately equal performance.44(https://en.wikipedia.org/wiki/LLVM#cite_note-44)

Components

LLVM has become an umbrella project containing multiple components.

Frontends

LLVM was originally written to be a replacement for the extant code generator in the GCC stack,45(https://en.wikipedia.org/wiki/LLVM#cite_note-45) and many of the GCC frontends were modified to work with it, resulting in the now-defunct LLVM-GCC suite. The modifications generally involved a GIMPLE-to-LLVM IR step so that LLVM optimizers and codegen could be used instead of GCC's GIMPLE system. Apple was a significant user of LLVM-GCC through Xcode 4.x (2013).46(https://en.wikipedia.org/wiki/LLVM#cite_note-46)[[47]](https://en.wikipedia.org/wiki/LLVM#cite_note-47) This use of the GCC frontend was considered a temporary measure which became mostly obsolete with the advent of LLVM/Clang's more modern, modular codebase and compilation speed.

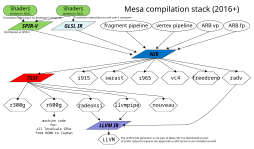
LLVM currently[*as of?*] supports compiling of Ada, C, C++, D, Delphi, Fortran, Haskell, Julia, Objective-C, Rust, and Swift using various frontends.

Widespread interest in LLVM has led to several efforts to develop new frontends for many languages. One such frontend is Clang, a newer compiler supporting C, C++, and Objective-C. Primarily supported by Apple, Clang is aimed at replacing the C/Objective-C compiler in the GCC system with a system that is more easily integrated with integrated development environments (IDEs) and has wider support for multithreading. Support for OpenMP directives has been included in Clang since release 3.8.48(https://en.wikipedia.org/wiki/LLVM#cite_note-48)

The Utrecht Haskell compiler can generate code for LLVM. While the generator was in early stages of development, in many cases it was more efficient than the C code generator.49(https://en.wikipedia.org/wiki/LLVM#cite_note-49) The Glasgow Haskell Compiler (GHC) backend uses LLVM and achieves a 30% speed-up of compiled code relative to native code compiling via GHC or C code generation followed by compiling, missing only one of the many optimizing techniques implemented by the GHC.50(https://en.wikipedia.org/wiki/LLVM#cite_note-50)

Many other components are in various stages of development, including, but not limited to, the Rust compiler, a Java bytecode frontend, a Common Intermediate Language (CIL) frontend, the MacRuby implementation of Ruby 1.9, various frontends for Standard ML, and a new graph coloring register allocator.[*citation needed*]

Intermediate representation



LLVM IR is used e.g., by radeonsi and by llvmpipe. Both are part of Mesa 3D.

The core of LLVM is the intermediate representation (IR), a low-level programming language similar to assembly. IR is a strongly typed reduced instruction set computer (RISC) instruction set which abstracts away most details of the target. For example, the calling convention is abstracted through `call` and `ret` instructions with explicit arguments. Also, instead of a fixed set of registers, IR uses an infinite set of temporaries of the form %0, %1, etc. LLVM supports three equivalent forms of IR: a human-readable assembly format,51(https://en.wikipedia.org/wiki/LLVM#cite_note-IRref-51) an in-memory format suitable for frontends, and a dense bitcode format for serializing. A simple "Hello, world!" program in the human-readable IR format:

```
@.str = internal constant [14 x i8] c"Hello, world\0A\00"

declare i32 @printf(ptr, ...)

define i32 @main(i32 %argc, ptr %argv) nounwind {
entry:
%tmp1 = getelementptr [14 x i8], ptr @.str, i32 0, i32 0
%tmp2 = call i32 (ptr, ...) @printf( ptr %tmp1 ) nounwind
ret i32 0
}
```

The many different conventions used and features provided by different targets mean that LLVM cannot truly produce a target-independent IR and retarget it without breaking some established rules. Examples of target dependence beyond what is explicitly mentioned in the documentation can be found in a 2011 proposal for "wordcode", a fully target-independent variant of LLVM IR intended for online distribution.52(https://en.wikipedia.org/wiki/LLVM#cite_note-52) A more practical example is PNaCl.53(https://en.wikipedia.org/wiki/LLVM#cite_note-53)

The LLVM project also introduces another type of intermediate representation named MLIR54(https://en.wikipedia.org/wiki/LLVM#cite_note-54) which helps build reusable and extensible compiler infrastructure by employing a plugin architecture named Dialect.55(https://en.wikipedia.org/wiki/LLVM#cite_note-55) It enables the use of higher-level information on the program structure in the process of optimization including polyhedral compilation.

Backends

At version 16, LLVM supports many instruction sets, including IA-32, x86-64, ARM, Qualcomm Hexagon, LoongArch, M68K, MIPS, NVIDIA Parallel Thread Execution (PTX, also named *NVPTX* in LLVM documentation), PowerPC, AMD TeraScale,56(https://en.wikipedia.org/wiki/LLVM#cite_note-56) most recent AMD GPUs (also named *AMDGPU* in LLVM documentation),57(https://en.wikipedia.org/wiki/LLVM#cite_note-57) SPARC, z/Architecture (also named *SystemZ* in LLVM documentation), and XCore.

Some features are not available on some platforms. Most features are present for IA-32, x86-64, z/Architecture, ARM, and PowerPC.58(https://en.wikipedia.org/wiki/LLVM#cite_note-58) RISC-V is supported as of version 7.

In the past, LLVM also supported other backends, fully or partially, including C backend, Cell SPU, mblaze (MicroBlaze),59(https://en.wikipedia.org/wiki/LLVM#cite_note-59) AMD R600, DEC/Compaq Alpha (Alpha AXP)60(https://en.wikipedia.org/wiki/LLVM#cite_note-60) and Nios2,61(https://en.wikipedia.org/wiki/LLVM#cite_note-llvm_2019-61) but that hardware is mostly obsolete, and LLVM developers decided the support and maintenance costs were no longer justified.[*citation needed*]

LLVM also supports WebAssembly as a target, enabling compiled programs to execute in WebAssembly-enabled environments such as Google Chrome / Chromium, Firefox, Microsoft Edge, Apple Safari or WAVM. LLVM-compliant WebAssembly compilers typically support mostly unmodified source code written in C, C++, D, Rust, Nim, Kotlin and several other languages.

The LLVM machine code (MC) subproject is LLVM's framework for translating machine instructions between textual forms and machine code. Formerly, LLVM relied on the system assembler, or one provided by a toolchain, to translate assembly into machine code. LLVM MC's integrated assembler supports most LLVM targets, including IA-32, x86-64, ARM, and ARM64. For some targets, including the various MIPS instruction sets, integrated assembly support is usable but still in the beta stage.[*citation needed*]

Linker

The lld subproject is an attempt to develop a built-in, platform-independent linker for LLVM.62(https://en.wikipedia.org/wiki/LLVM#cite_note-lld-home-page-62) lld aims to remove dependence on a third-party linker. As of May 2017, lld supports ELF, PE/COFF, Mach-O, and WebAssembly63(https://en.wikipedia.org/wiki/LLVM#cite_note-63) in descending order of completeness. lld is faster than both flavors of GNU ld.[*citation needed*]

Unlike the GNU linkers, lld has built-in support for link-time optimization (LTO). This allows for faster code generation as it bypasses the use of a linker plugin, but on the other hand prohibits interoperability with other flavors of LTO.64(https://en.wikipedia.org/wiki/LLVM#cite_note-64)

C++ Standard Library

The LLVM project includes an implementation of the C++ Standard Library named libc++, dual-licensed under the MIT License and the UIUC license.65(https://en.wikipedia.org/wiki/LLVM#cite_note-65)

Since v9.0.0, it was relicensed to the Apache License 2.0 with LLVM Exceptions.3(https://en.wikipedia.org/wiki/LLVM#cite_note-License-3)

Polly

This implements a suite of cache-locality optimizations as well as auto-parallelism and vectorization using a polyhedral model.66(https://en.wikipedia.org/wiki/LLVM#cite_note-66)

Debugger

Main article: LLDB (debugger)

C Standard Library

llvm-libc is an incomplete, upcoming, ABI independent C standard library designed by and for the LLVM project.67(https://en.wikipedia.org/wiki/LLVM#cite_note-67)

Dialects

In MLIR, a **dialect** defines a self-contained namespace of operations, types, attributes, and other constructs. Dialects are the primary mechanism for extensibility, allowing developers to introduce domain-specific abstractions while maintaining compatibility within the broader MLIR framework. Each operation within a dialect is identified by a unique name and may include optional operands, results, attributes, and regions. Operands and results follow the static single-assignment form (SSA), and each result is associated with a type. Attributes represent compile-time metadata, such as constant values. Regions consist of ordered blocks, each of which may take input arguments and contain a sequence of nested operations.13(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-13) While MLIR is designed around SSA, it avoids traditional PHI nodes by using block arguments in conjunction with the operands of control-flow operations to model value merging.14(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-14)

The general syntax for an operation is the following:

```
%res:2 = "mydialect.morph"(%input#3) ({
^bb0(%arg0: !mydialect<"custom_type"> loc("mysource.cc":10:8)):
// nested operations
}) { some.attribute = true, other_attribute = 1.5 }
: (!mydialect<"custom_type">) -> (!mydialect<"other_type">, !mydialect<"other_type">)
loc(callsite("foo" at "mysource.cc":10:8))
```

This operation, named `morph`, belongs to the `mydialect` dialect. It takes one input operand (`%input#3`) of type `custom_type` and produces two output values of type `other_type`. The operation includes two attributes- `some.attribute` and `other_attribute` -and contains a region with a single block (`^bb0`) that accepts one argument. The `loc` keyword specifies source-level location information, which can be used for debugging or diagnostic reporting.15(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-15)

The syntax of operations, types and attributes can also be customized according to the user preferences by implementing proper parsing and printing functions within the operation definition.16(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-16)

Core dialects

The MLIR dialects ecosystem is open and extensible, allowing end-users to define new dialects that capture the semantics of specific computational domains. At the same time, the MLIR codebase provides a variety of built-in dialects that address common patterns found in intermediate representations. These core dialects are designed to be self-contained and interoperable, making them suitable for reuse across different compiler stacks.17(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-:2-17)

For example, the `arith` dialect includes basic mathematical operations over integers and floating-point types, while the `memref` dialect provides operations for memory allocation and access. Control-flow abstractions are handled by dialects such as `affine`, which supports affine loop nests suitable for polyhedral optimization, and `scf`, which provides structured control flow using constructs like `for`, `if`, and `while`. The `func` dialect supports function definitions and calls, while the `gpu` dialect introduces primitives for GPU programming models. Additionally, the `tosa` dialect defines a portable and quantization-friendly operator set for machine learning inference. Finally, the `llvm` dialect provides a one-to-one mapping to LLVM IR, enabling seamless lowering to LLVM's backend and reuse of its optimization and code generation infrastructure.4(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-mlir-users-4)

The following code defines a function that takes two floating point matrices and performs the sum between the values at the same positions:

func.func @matrix_add(%arg0: memref<10x20xf32>, %arg1: memref<10x20xf32>) -> memref<10x20xf32> {
%result = memref.alloc() : memref<10x20xf32>

```
affine.for %i = 0 to 10 {
    affine.for %j = 0 to 20 {
        %lhs = memref.load %arg0[%i, %j] : memref<10x20xf32>
        %rhs = memref.load %arg1[%i, %j] : memref<10x20xf32>
        %sum = arith.addf %lhs, %rhs : f32
        memref.store %sum, %result[%i, %j] : memref<10x20xf32>
    }
}

func.return %result : memref<10x20xf32>
```

}

Although different dialects may be used to express similar computations, the level of abstraction and the intended compilation flow may vary. In the example above, the `affine` dialect enables polyhedral analysis and optimizations, while the `memref` and `arith` dialects express memory and arithmetic operations, respectively.17(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-:2-17)

Operation definition specification

The operations of a dialect can be defined using the C++ language, but also in a more convenient and robust way by using the Operation definition specification (ODS).18(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-:5-18) By using TableGen, the C++ code for declarations and definitions can be then automatically generated.19(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-19)

The autogenerated code can include parsing and printing methods – which are based on a simple string mapping the structure of desired textual representation – together with all the boilerplate code for accessing fields and perform common actions such verification of the semantics of each operation, canonicalization or folding.20(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-:3-20)

The same declaration mechanism can be used also for types and attributes, which are the other two categories of elements constituting a dialect.20(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-:3-20)

The following example illustrates how to specify the *assembly format* of an operation expecting a variadic number of operands and producing zero results. The textual representation consists in the optional list of attributes, followed by the optional list of operands, a colon, and types of the operands.18(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-:5-18)

let assemblyFormat = "attr-dict ($operands$' :$type$(operands))?";

Transformations

Transformations can always be performed directly on the IR, without having to rely on built-in coordination mechanisms. However, in order to ease both implementation and maintenance, MLIR provides an infrastructure for IR rewriting that is composed by different rewrite drivers. Each driver receives a set of objects named *patterns*, each of which has its own internal logic to match operations with certain properties. When an operation is matched, the rewrite process is performed and the IR is modified according to the logic within the pattern.21(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-:1-21)

Dialect conversion driver

This driver operates according to the *legality* of existing operations, meaning that the driver receives a set of rules determining which operations have to be considered *illegal* and expects the patterns to match and convert them into *legal* ones. The logic behind those rules can be arbitrarily complex: it may be based just on the dialect to which the operations belong, but can also inspect more specific properties such as attributes or nested operations.22(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-:4-22)

As the names suggests, this driver is typically used for converting the operations of a dialect into operations belonging to a different one. In this scenario, the whole source dialect would be marked as illegal, the destination one as legal, and patterns for the source dialect operations would be provided. The dialect conversion framework also provides support for type conversion, which has to be performed on operands and results to convert them to the type system of the destination dialect.22(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-:4-22)

MLIR allows for multiple conversion paths to be taken. Considering the example about the sum of matrices, a possible lowering strategy may be to generate for-loops belonging to the *scf* dialect, obtaining code to be executed on CPUs:

#map = affine_map<(d0, d1) -> (d0, d1)>
module {
func.func @avg(%arg0: memref<10x20xf32>, %arg1: memref<10x20xf32>) -> memref<10x20xf32> {
%alloc = memref.alloc() : memref<10x20xf32>
%c0 = arith.constant 0 : index
%c10 = arith.constant 10 : index
%c1 = arith.constant 1 : index

```
    scf.for %arg2 = %c0 to %c10 step %c1 {
        %c0_0 = arith.constant 0 : index
        %c20 = arith.constant 20 : index
        %c1_1 = arith.constant 1 : index

        scf.for %arg3 = %c0_0 to %c20 step %c1_1 {
            %0 = memref.load %arg0[%arg2, %arg3] : memref<10x20xf32>
            %1 = memref.load %arg1[%arg2, %arg3] : memref<10x20xf32>
            %2 = arith.addf %0, %1 : f32
            memref.store %2, %alloc[%arg2, %arg3] : memref<10x20xf32>
        }
    }

    return %alloc : memref<10x20xf32>
}
```

}

Another possible strategy, however, could have been to use the *gpu* dialect to generate code for GPUs:

#map = affine_map<(d0, d1) -> (d0, d1)>
module {
func.func @avg(%arg0: memref<10x20xf32>, %arg1: memref<10x20xf32>) -> memref<10x20xf32> {

%alloc = memref.alloc() : memref<10x20xf32>
%c0 = arith.constant 0 : index
%c10 = arith.constant 10 : index
%0 = arith.subi %c10, %c0 : index
%c1 = arith.constant 1 : index
%c0_0 = arith.constant 0 : index
%c20 = arith.constant 20 : index
%1 = arith.subi %c20, %c0_0 : index
%c1_1 = arith.constant 1 : index
%c1_2 = arith.constant 1 : index

```
    gpu.launch blocks(%arg2, %arg3, %arg4) in (%arg8 = %0, %arg9 = %c1_2, %arg10 = %c1_2) threads(%arg5, %arg6, %arg7) in (%arg11 = %1, %arg12 = %c1_2, %arg13 =
%c1_2) {
        %2 = arith.addi %c0, %arg2 : index
        %3 = arith.addi %c0_0, %arg5 : index
        %4 = memref.load %arg0[%2, %3] : memref<10x20xf32>
        %5 = memref.load %arg1[%2, %3] : memref<10x20xf32>
        %6 = arith.addf %4, %5 : f32
        memref.store %4, %alloc[%2, %3] : memref<10x20xf32>
        gpu.terminator
    }

    return %alloc : memref<10x20xf32>
}
```

}

Greedy pattern rewrite driver

The driver greedily applies the provided patterns according to their benefit, until a fixed point is reached or the maximum number of iterations is reached. The benefit of a pattern is self-attributed. In case of equalities, the relative order within the patterns list is used.21(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-:1-21)

Traits and interfaces

MLIR allows to apply existing optimizations (e.g., common subexpression elimination, loop-invariant code motion) on custom dialects by means of traits and interfaces. These two mechanisms enable transformation passes to operate on operations without knowing their actual implementation, relying only on some properties that traits or interfaces provide.23(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-:0-23)[[24]](https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-:6-24)

Traits are meant to be attached to operations without requiring any additional implementation. Their purpose is to indicate that the operation satisfies certain properties (e.g. having exactly two operands).23(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-:0-23) Interfaces, instead, represent a more powerful tool through which the operation can be queried about some specific aspect, whose value may change between instances of the same kind of operation. An example of interface is the representation of memory effects: each operation that operates on memory may have such interface attached, but the actual effects may depend on the actual operands (e.g., a function call with arguments possibly being constants or references to memory).24(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-:6-24)

Applications

The freedom in modeling intermediate representations enables MLIR to be used in a wide range of scenarios. This includes traditional programming languages,25(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-25) but also high-level synthesis,26(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-26)[[27]](https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-27) quantum computing28(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-28) and homomorphic encryption.29(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-29)[[30]](https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-30)[[31]](https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-31) Machine learning applications also take advantage of built-in polyhedral compilation techniques, together with dialects targeting accelerators and other heterogeneous systems.32(https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-32)[[33]](https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-33)[[34]](https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-34)[[35]](https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-35)[[36]](https://en.wikipedia.org/wiki/MLIR_\(software\)#cite_note-36)

Features

One of the design goals of cgroups is to provide a unified interface to many different use cases, from controlling single processes (by using nice, for example) to full operating system-level virtualization (as provided by OpenVZ, Linux-VServer or LXC, for example). Cgroups provides:

Resource limiting

groups can be set not to exceed a configured memory limit, which also includes the file system cache,10(https://en.wikipedia.org/wiki/Cgroups#cite_note-10)[[11]](https://en.wikipedia.org/wiki/Cgroups#cite_note-ols-memcg-11) I/O bandwidth limit,12(https://en.wikipedia.org/wiki/Cgroups#cite_note-12) CPU quota limit,13(https://en.wikipedia.org/wiki/Cgroups#cite_note-13) CPU set limit,14(https://en.wikipedia.org/wiki/Cgroups#cite_note-14) or maximum open files.15(https://en.wikipedia.org/wiki/Cgroups#cite_note-15)

Prioritization

some groups may get a larger share of CPU utilization16(https://en.wikipedia.org/wiki/Cgroups#cite_note-16) or disk I/O throughput17(https://en.wikipedia.org/wiki/Cgroups#cite_note-17)

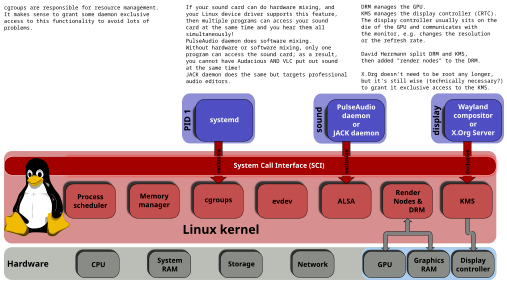
Accounting

measures a group's resource usage, which may be used, for example, for billing purposes18(https://en.wikipedia.org/wiki/Cgroups#cite_note-lf-hansen-18)

Control

freezing groups of processes, their checkpointing and restarting18(https://en.wikipedia.org/wiki/Cgroups#cite_note-lf-hansen-18)

Use



As an example of indirect usage, systemd assumes exclusive access to the cgroups facility

A control group (abbreviated as cgroup) is a collection of processes that are bound by the same criteria and associated with a set of parameters or limits. These groups can be hierarchical, meaning that each group inherits limits from its parent group. The kernel provides access to multiple controllers (also called subsystems) through the cgroup interface;3(https://en.wikipedia.org/wiki/Cgroups#cite_note-lwn-notes-3) for example, the "memory" controller limits memory use, "cpuacct" accounts CPU usage, etc.

Control groups can be used in multiple ways:

- By accessing the cgroup virtual file system manually.
- By creating and managing groups on the fly using tools like `cgcreate`, `cgexec`, and `cgclassify` (from `libcgroup`).
- Through the "rules engine daemon" that can automatically move processes of certain users, groups, or commands to cgroups as specified in its configuration.
- Indirectly through other software that uses cgroups, such as Docker, Firejail, LXC,19(https://en.wikipedia.org/wiki/Cgroups#cite_note-19) libvirt, systemd, Open Grid Scheduler/Grid Engine,20(https://en.wikipedia.org/wiki/Cgroups#cite_note-20) and Google's developmentally defunct lmctfy.

The Linux kernel documentation contains some technical details of the setup and use of control groups version 121(https://en.wikipedia.org/wiki/Cgroups#cite_note-v1doc-21) and version 2.1(https://en.wikipedia.org/wiki/Cgroups#cite_note-v2doc-1)

Interfaces

Both versions of cgroup act through a pseudo-filesystem (`cgroup` for v1 and `cgroup2` for v2). Like all filesystems they can be mounted on any path, but the general convention is to mount one of the versions (generally v2) on `/sys/fs/cgroup` under the sysfs default location of `/sys`. As mentioned before the two cgroup versions can be active at the same time; this too applies to the filesystems so long as they are mounted to a different path.21(https://en.wikipedia.org/wiki/Cgroups#cite_note-v1doc-21)[[1]](https://en.wikipedia.org/wiki/Cgroups#cite_note-v2doc-1) For the description below we assume a setup where the v2 hierarchy lies in `/sys/fs/cgroup`. The v1 hierarchy, if ever required, will be mounted at a different location.

At initialization cgroup2 should have no defined control groups except the top-level one. In other words, `/sys/fs/cgroup` should have no directories, only a number of files that control the system as a whole. At this point, running `ls /sys/fs/cgroup` could list the following on one example system:

- `cgroup.controllers`
- `cgroup.max.depth`
- `cgroup.max.descendants`
- `cgroup.pressure`
- `cgroup.procs`
- `cgroup.stat`
- `cgroup.subtree_control`
- `cgroup.threads`
- `cpu.pressure`
- `cpuset.cpus.effective`
- `cpuset.cpus.isolated`
- `cpuset.mems.effective`
- `cpu.stat`
- `cpu.stat.local`
- `io.cost.model`
- `io.cost.qos`
- `io.pressure`
- `io.prio.class`
- `io.stat`
- `irq.pressure`
- `memory.numa_stat`
- `memory.pressure`
- `memory.reclaim`
- `memory.stat`
- `memory.zswap.writeback`
- `misc.capacity`
- `misc.current`
- `misc.peak`

These files are named according to the controllers that handle them. For example, `cgroup.*` deal with the cgroup system itself and `memory.*` deal with the memory subsystem. Example: to request the kernel to 1 gigabyte of memory from anywhere in the system, one can run `echo "1G swappiness=50" > /sys/fs/cgroup/memory.reclaim`.1(https://en.wikipedia.org/wiki/Cgroups#cite_note-v2doc-1)

To create a subgroup, one simply creates a new directory under an existing group (including the top-level one). The files corresponding to available controls for this group are automatically created.1(https://en.wikipedia.org/wiki/Cgroups#cite_note-v2doc-1) For example, running `mkdir /sys/fs/cgroup/example; ls /sys/fs/cgroup/example` would produce a list of files largely similar to the one above, but with noticeable changes. On one example system, these files are added:

- `cgroup.events`
- `cgroup.freeze`
- `cgroup.kill`
- `cgroup.type`
- `cpu.idle`

- `cpu.max`
- `cpu.max.burst`
- `cpu.pressure`
- `cpu.uclamp.max`
- `cpu.uclamp.min`
- `cpu.weight`
- `cpu.weight.nice`
- `memory.current`
- `memory.events`
- `memory.events.local`
- `memory.high`
- `memory.low`
- `memory.max`
- `memory.min`
- `memory.oom.group`
- `memory.peak`
- `memory.swap.current`
- `memory.swap.events`
- `memory.swap.high`
- `memory.swap.max`
- `memory.swap.peak`
- `memory.zswap.current`
- `memory.zswap.max`
- `pids.current`
- `pids.events`
- `pids.events.local`
- `pids.max`
- `pids.peak`

These changes are not unexpected because some controls and statistics only make sense on a subset of processes (e.g. nice level being the CPU priority of processes relative to the rest of the system).1(https://en.wikipedia.org/wiki/Cgroups#cite_note-v2doc-1)

Processes are assigned to subgroups by writing to `/proc/<PID>/cgroup`. The cgroup a process is in can be found by reading the same file.1(https://en.wikipedia.org/wiki/Cgroups#cite_note-v2doc-1)

---

kernel data structure for DRBD-9

This describes the in kernel data structure for DRBD-9. Starting with Linux v3.14 we are reorganizing DRBD to use this data structure.

Basic Data Structure

A node has a number of DRBD resources. Each such resource has a number of devices (aka volumes) and connections to other nodes (“peer nodes”). Each DRBD device is represented by a block device locally.

The DRBD objects are interconnected to form a matrix as depicted below; a drbd_peer_device object sits at each intersection between a drbd_device and a drbd_connection:

```
/--------------+---------------+.....+---------------
| resource | device | | device |
+--------------+---------------+.....+---------------+
| connection | peer_device | | peer_device |
+--------------+---------------+.....+---------------+
: : : : :
: : : : :
+--------------+---------------+.....+---------------+
| connection | peer_device | | peer_device |
--------------+--------------+.....+--------------/
```

In this table, horizontally, devices can be accessed from resources by their volume number. Likewise, peer_devices can be accessed from connections by their volume number. Objects in the vertical direction are connected by double linked lists. There are back pointers from peer_devices to their connections a devices, and from connections and devices to their resource.

All resources are in the drbd_resources double-linked list. In addition, all devices can be accessed by their minor device number via the drbd_devices idr.

The drbd_resource, drbd_connection, and drbd_device objects are reference counted. The peer_device objects only serve to establish the links between devices and connections; their lifetime is determined by the lifetime of the device and connection which they reference.

Exchanging pixel buffers

As originally designed, the Linux graphics subsystem had extremely limited support for sharing pixel-buffer allocations between processes, devices, and subsystems. Modern systems require extensive integration between all three classes; this document details how applications and kernel subsystems should approach this sharing for two-dimensional image data.

It is written with reference to the DRM subsystem for GPU and display devices, V4L2 for media devices, and also to Vulkan, EGL and Wayland, for userspace support, however any other subsystems should also follow this design and advice.

Glossary of terms

image:

Conceptually a two-dimensional array of pixels. The pixels may be stored in one or more memory buffers. Has width and height in pixels, pixel format and modifier (implicit or explicit).

row:

A span along a single y-axis value, e.g. from co-ordinates (0,100) to (200,100).

scanline:

Synonym for row.

column:

A span along a single x-axis value, e.g. from co-ordinates (100,0) to (100,100).

memory buffer:

A piece of memory for storing (parts of) pixel data. Has stride and size in bytes and at least one handle in some API. May contain one or more planes.

plane:

A two-dimensional array of some or all of an image's color and alpha channel values.

pixel:

A picture element. Has a single color value which is defined by one or more color channels values, e.g. R, G and B, or Y, Cb and Cr. May also have an alpha value as an additional channel.

pixel data:

Bytes or bits that represent some or all of the color/alpha channel values of a pixel or an image. The data for one pixel may be spread over several planes or memory buffers depending on format and modifier.

color value:

A tuple of numbers, representing a color. Each element in the tuple is a color channel value.

color channel:

One of the dimensions in a color model. For example, RGB model has channels R, G, and B. Alpha channel is sometimes counted as a color channel as well.

pixel format:

A description of how pixel data represents the pixel's color and alpha values.

modifier:

A description of how pixel data is laid out in memory buffers.

alpha:

A value that denotes the color coverage in a pixel. Sometimes used for translucency instead.

stride:

A value that denotes the relationship between pixel-location co-ordinates and byte-offset values. Typically used as the byte offset between two pixels at the start of vertically-consecutive tiling blocks. For linear layouts, the byte offset between two vertically-adjacent pixels. For non-linear formats the stride must be computed in a consistent way, which usually is done as-if the layout was linear.

pitch:

Synonym for stride.

Formats and modifiers

Each buffer must have an underlying format. This format describes the color values provided for each pixel. Although each subsystem has its own format descriptions (e.g. V4L2 and fbdev), the `DRM_FORMAT_*` tokens should be reused wherever possible, as they are the standard descriptions used for interchange. These tokens are described in the `drm_fourcc.h` file, which is a part of DRM's uAPI.

Each `DRM_FORMAT_*` token describes the translation between a pixel co-ordinate in an image, and the color values for that pixel contained within its memory buffers. The number and type of color channels are described: whether they are RGB or YUV, integer or floating-point, the size of each channel and their locations within the pixel memory, and the relationship between color planes.

For example, `DRM_FORMAT_ARGB8888` describes a format in which each pixel has a single 32-bit value in memory. Alpha, red, green, and blue, color channels are available at 8-bit precision per channel, ordered respectively from most to least significant bits in little-endian storage. `DRM_FORMAT_*` is not affected by either CPU or device endianness; the byte pattern in memory is always as described in the format definition, which is usually little-endian.

As a more complex example, `DRM_FORMAT_NV12` describes a format in which luma and chroma YUV samples are stored in separate planes, where the chroma plane is stored at half the resolution in both dimensions (i.e. one U/V chroma sample is stored for each 2x2 pixel grouping).

Format modifiers describe a translation mechanism between these per-pixel memory samples, and the actual memory storage for the buffer. The most straightforward modifier is `DRM_FORMAT_MOD_LINEAR`, describing a scheme in which each plane is laid out row-sequentially, from the top-left to the bottom-right corner. This is considered the baseline interchange format, and most convenient for CPU access.

Modern hardware employs much more sophisticated access mechanisms, typically making use of tiled access and possibly also compression. For example, the `DRM_FORMAT_MOD_VIVANTE_TILED` modifier describes memory storage where pixels are stored in 4x4 blocks arranged in row-major ordering, i.e. the first tile in a plane stores pixels (0,0) to (3,3) inclusive, and the second tile in a plane stores pixels (4,0) to (7,3) inclusive.

Some modifiers may modify the number of planes required for an image; for example, the `I915_FORMAT_MOD_Y_TILED_CCS` modifier adds a second plane to RGB formats in which it stores data about the status of every tile, notably including whether the tile is fully populated with pixel data, or can be expanded from a single solid color.

These extended layouts are highly vendor-specific, and even specific to particular generations or configurations of devices per-vendor. For this reason, support of modifiers must be explicitly enumerated and negotiated by all users in order to ensure a compatible and optimal pipeline, as discussed below.

Dimensions and size

Each pixel buffer must be accompanied by logical pixel dimensions. This refers to the number of unique samples which can be extracted from, or stored to, the underlying memory storage. For example, even though a 1920x1080 `DRM_FORMAT_NV12` buffer has a luma plane containing 1920x1080 samples for the Y component, and 960x540 samples for the U and V components, the overall buffer is still described as having dimensions of 1920x1080.

The in-memory storage of a buffer is not guaranteed to begin immediately at the base address of the underlying memory, nor is it guaranteed that the memory storage is tightly clipped to either dimension.

Each plane must therefore be described with an `offset` in bytes, which will be added to the base address of the memory storage before performing any per-pixel calculations. This may be used to combine multiple planes into a single memory buffer; for example, `DRM_FORMAT_NV12` may be stored in a single memory buffer where the luma plane's storage begins immediately at the start of the

buffer with an offset of 0, and the chroma plane's storage follows within the same buffer beginning from the byte offset for that plane.

Each plane must also have a `stride` in bytes, expressing the offset in memory between two contiguous row. For example, a `DRM_FORMAT_MOD_LINEAR` buffer with dimensions of 1000x1000 may have been allocated as if it were 1024x1000, in order to allow for aligned access patterns. In this case, the buffer will still be described with a width of 1000, however the stride will be `1024 * bpp`, indicating that there are 24 pixels at the positive extreme of the x axis whose values are not significant.

Buffers may also be padded further in the y dimension, simply by allocating a larger area than would ordinarily be required. For example, many media decoders are not able to natively output buffers of height 1080, but instead require an effective height of 1088 pixels. In this case, the buffer continues to be described as having a height of 1080, with the memory allocation for each buffer being increased to account for the extra padding.

Enumeration

Every user of pixel buffers must be able to enumerate a set of supported formats and modifiers, described together. Within KMS, this is achieved with the `IN_FORMATS` property on each DRM plane, listing the supported DRM formats, and the modifiers supported for each format. In userspace, this is supported through the EGL_EXT_image_dma_buf_import_modifiers extension entrypoints for EGL, the VK_EXT_image_drm_format_modifier extension for Vulkan, and the zwp_linux_dmabuf_v1 extension for Wayland.

Each of these interfaces allows users to query a set of supported format+modifier combinations.

Negotiation

It is the responsibility of userspace to negotiate an acceptable format+modifier combination for its usage. This is performed through a simple intersection of lists. For example, if a user wants to use Vulkan to render an image to be displayed on a KMS plane, it must:

> - query KMS for the `IN_FORMATS` property for the given plane
> - query Vulkan for the supported formats for its physical device, making sure to pass the `VkImageUsageFlagBits` and `VkImageCreateFlagBits` corresponding to the intended rendering use
> - intersect these formats to determine the most appropriate one
> - for this format, intersect the lists of supported modifiers for both KMS and Vulkan, to obtain a final list of acceptable modifiers for that format

This intersection must be performed for all usages. For example, if the user also wishes to encode the image to a video stream, it must query the media API it intends to use for encoding for the set of modifiers it supports, and additionally intersect against this list.

If the intersection of all lists is an empty list, it is not possible to share buffers in this way, and an alternate strategy must be considered (e.g. using CPU access routines to copy data between the different uses, with the corresponding performance cost).

The resulting modifier list is unsorted; the order is not significant.

Allocation

Once userspace has determined an appropriate format, and corresponding list of acceptable modifiers, it must allocate the buffer. As there is no universal buffer-allocation interface available at either kernel or userspace level, the client makes an arbitrary choice of allocation interface such as Vulkan, GBM, or a media API.

Each allocation request must take, at a minimum: the pixel format, a list of acceptable modifiers, and the buffer's width and height. Each API may extend this set of properties in different ways, such as allowing allocation in more than two dimensions, intended usage patterns, etc.

The component which allocates the buffer will make an arbitrary choice of what it considers the 'best' modifier within the acceptable list for the requested allocation, any padding required, and further properties of the underlying memory buffers such as whether they are stored in system or device-specific memory, whether or not they are physically contiguous, and their cache mode. These properties of the memory buffer are not visible to userspace, however the `dma-heaps` API is an effort to address this.

After allocation, the client must query the allocator to determine the actual modifier selected for the buffer, as well as the per-plane offset and stride. Allocators are not permitted to vary the format in use, to select a modifier not provided within the acceptable list, nor to vary the pixel dimensions other than the padding expressed through offset, stride, and size.

Communicating additional constraints, such as alignment of stride or offset, placement within a particular memory area, etc, is out of scope of dma-buf, and is not solved by format and modifier tokens.

Import

To use a buffer within a different context, device, or subsystem, the user passes these parameters (format, modifier, width, height, and per-plane offset and stride) to an importing API.

Each memory buffer is referred to by a buffer handle, which may be unique or duplicated within an image. For example, a `DRM_FORMAT_NV12` buffer may have the luma and chroma buffers combined into a single memory buffer by use of the per-plane offset parameters, or they may be completely separate allocations in memory. For this reason, each import and allocation API must provide a separate handle for each plane.

Each kernel subsystem has its own types and interfaces for buffer management. DRM uses GEM buffer objects (BOs), V4L2 has its own references, etc. These types are not portable between contexts, processes, devices, or subsystems.

To address this, `dma-buf` handles are used as the universal interchange for buffers. Subsystem-specific operations are used to export native buffer handles to a `dma-buf` file descriptor, and to import those file descriptors into a native buffer handle. dma-buf file descriptors can be transferred between contexts, processes, devices, and subsystems.

For example, a Wayland media player may use V4L2 to decode a video frame into a `DRM_FORMAT_NV12` buffer. This will result in two memory planes (luma and chroma) being dequeued by the user from V4L2. These planes are then exported to one dma-buf file descriptor per plane, these descriptors are then sent along with the metadata (format, modifier, width, height, per-plane offset and stride) to the Wayland server. The Wayland server will then import these file descriptors as an EGLImage for use through EGL/OpenGL (ES), a VkImage for use through Vulkan, or a KMS framebuffer object; each of these import operations will take the same metadata and convert the dma-buf file descriptors into their native buffer handles.

Having a non-empty intersection of supported modifiers does not guarantee that import will succeed into all consumers; they may have constraints beyond those implied by modifiers which must be satisfied.

Implicit modifiers

The concept of modifiers post-dates all of the subsystems mentioned above. As such, it has been retrofitted into all of these APIs, and in order to ensure backwards compatibility, support is needed for drivers and userspace which do not (yet) support modifiers.

As an example, GBM is used to allocate buffers to be shared between EGL for rendering and KMS for display. It has two entrypoints for allocating buffers: `gbm_bo_create` which only takes the format, width, height, and a usage token, and `gbm_bo_create_with_modifiers` which extends this with a list of modifiers.

In the latter case, the allocation is as discussed above, being provided with a list of acceptable modifiers that the implementation can choose from (or fail if it is not possible to allocate within those constraints). In the former case where modifiers are not provided, the GBM implementation must make its own choice as to what is likely to be the 'best' layout. Such a choice is entirely implementation-specific: some will internally use tiled layouts which are not CPU-accessible if the implementation decides that is a good idea through whatever heuristic. It is the implementation's responsibility to ensure that this choice is appropriate.

To support this case where the layout is not known because there is no awareness of modifiers, a special `DRM_FORMAT_MOD_INVALID` token has been defined. This pseudo-modifier declares that the layout is not known, and that the driver should use its own logic to determine what the underlying layout may be.

Note

`DRM_FORMAT_MOD_INVALID` is a non-zero value. The modifier value zero is `DRM_FORMAT_MOD_LINEAR`, which is an explicit guarantee that the image has the linear layout. Care and attention should be taken to ensure that zero as a default value is not mixed up with either no modifier or the linear modifier. Also note that in some APIs the invalid modifier value is specified with an out-of-band flag, like in `DRM_IOCTL_MODE_ADDFB2`.

There are four cases where this token may be used:

- during enumeration, an interface may return `DRM_FORMAT_MOD_INVALID`, either as the sole member of a modifier list to declare that explicit modifiers are not supported, or as part of a larger list to declare that implicit modifiers may be used
- during allocation, a user may supply `DRM_FORMAT_MOD_INVALID`, either as the sole member of a modifier list (equivalent to not supplying a modifier list at all) to declare that explicit modifiers are not supported and must not be used, or as part of a larger list to declare that an allocation using implicit modifiers is acceptable
- in a post-allocation query, an implementation may return `DRM_FORMAT_MOD_INVALID` as the modifier of the allocated buffer to declare that the underlying layout is implementation-defined and that an explicit modifier description is not available; per the above rules, this may only be returned when the user has included `DRM_FORMAT_MOD_INVALID` as part of the list of acceptable modifiers, or not provided a list
- when importing a buffer, the user may supply `DRM_FORMAT_MOD_INVALID` as the buffer modifier (or not supply a modifier) to indicate that the modifier is unknown for whatever reason; this is only acceptable when the buffer has not been allocated with an explicit modifier

It follows from this that for any single buffer, the complete chain of operations formed by the producer and all the consumers must be either fully implicit or fully explicit. For example, if a user wishes to allocate a buffer for use between GPU, display, and media, but the media API does not support modifiers, then the user **must not** allocate the buffer with explicit modifiers and attempt to import the buffer into the media API with no modifier, but either perform the allocation using implicit modifiers, or allocate the buffer for media use separately and copy between the two buffers.

As one exception to the above, allocations may be 'upgraded' from implicit to explicit modifiers. For example, if the buffer is allocated with `gbm_bo_create` (taking no modifiers), the user may then query the modifier with `gbm_bo_get_modifier` and then use this modifier as an explicit modifier token if a valid modifier is returned.

When allocating buffers for exchange between different users and modifiers are not available, implementations are strongly encouraged to use `DRM_FORMAT_MOD_LINEAR` for their allocation, as this is the universal baseline for exchange. However, it is not guaranteed that this will result in the correct interpretation of buffer content, as implicit modifier operation may still be subject to driver-specific heuristics.

Any new users - userspace programs and protocols, kernel subsystems, etc - wishing to exchange buffers must offer interoperability through dma-buf file descriptors for memory planes, DRM format tokens to describe the format, DRM format modifiers to describe the layout in memory, at least width and height for dimensions, and at least offset and stride for each memory plane.

---

these tie directly to our step rotation functions

Zoned Filesystems

For zoned file systems, the following attributes are exposed in:

> /sys/fs/xfs//zoned/
>
> max_open_zones (Min: 1 Default: Varies Max: UINTMAX)
>
> This read-only attribute exposes the maximum number of open zones available for data placement. The value is determined at mount time and is limited by the capabilities of the backing zoned device, file system size and the max_open_zones mount option.
>
> zonegc_low_space (Min: 0 Default: 0 Max: 100)
>
> Define a percentage for how much of the unused space that GC should keep available for writing. A high value will reclaim more of the space occupied by unused blocks, creating a larger buffer against write bursts at the cost of increased write amplification. Regardless of this value, garbage collection will always aim to free a minimum amount of blocks to keep max_open_zones open for data placement purposes.

A **sliding window protocol** is a feature of packet-based data transmission protocols. Sliding window protocols are used where reliable in-order delivery of packets is required, such as in the data link layer (OSI layer 2) as well as in the Transmission Control Protocol (i.e., TCP windowing). They are also used to improve efficiency when the channel may include high latency.

Packet-based systems are based on the idea of sending a batch of data, the *packet*, along with additional data that allows the receiver to ensure it was received correctly, perhaps a checksum. The paradigm is similar to a window sliding sideways to allow entry of fresh packets and reject the ones that have already been acknowledged. When the receiver verifies the data, it sends an acknowledgment signal, or ACK, back to the sender to indicate it can send the next packet. In a simple automatic repeat request protocol (ARQ), the sender stops after every packet and waits for the receiver to ACK. This ensures packets arrive in the correct order, as only one may be sent at a time.

The time that it takes for the ACK signal to be received may represent a significant amount of time compared to the time needed to send the packet. In this case, the overall throughput may be much lower than theoretically possible. To address this, sliding window protocols allow a selected number of packets, the *window*, to be sent without having to wait for an ACK. Each packet receives a sequence number, and the ACKs send back that number. The protocol keeps track of which packets have been ACKed, and when they are received, sends more packets. In this way, the window *slides* along the stream of packets making up the transfer.

Sliding windows are a key part of many protocols. It is a key part of the TCP protocol, which inherently allows packets to arrive out of order, and is also found in many file transfer protocols like UUCP-g and ZMODEM as a way of improving efficiency compared to non-windowed protocols like XMODEM. See also SEAlink.

Basic concept

Conceptually, each portion of the transmission (packets in most data link layers, but bytes in TCP) is assigned a unique consecutive sequence number, and the receiver uses the numbers to place received packets in the correct order, discarding duplicate packets and identifying missing ones. The problem with this is that there is no limit on the size of the sequence number that can be required.

By placing limits on the number of packets that can be transmitted or received at any given time, a sliding window protocol allows an unlimited number of packets to be communicated using fixed-size sequence numbers. The term *window* on the transmitter side represents the logical boundary of the total number of packets yet to be acknowledged by the receiver. The receiver informs the transmitter in each acknowledgment packet of the current maximum receiver buffer size (window boundary). The TCP header uses a 16-bit field to report the receiver window size to the sender. Therefore, the largest window that can be used is 216 = 64 kilobytes.

In slow-start mode, the transmitter starts with a low packet count and increases the number of packets in each transmission after receiving acknowledgment packets from the receiver. For every ACK packet received, the window slides by one packet (logically) to transmit one new packet. When the window threshold is reached, the transmitter sends one packet for each ACK packet received.

If the window limit is 10 packets, then in slow start mode, the transmitter may start transmitting one packet, followed by two packets (before transmitting two packets, one packet ACK has to be received), followed by three packets and so on until 10 packets. But after reaching 10 packets, further transmissions are restricted to one packet transmitted for one ACK packet received. In a simulation, this appears as if the window is moving by one packet distance for every ACK packet received. On the receiver side, the window moves one packet for every packet received.

The sliding window method ensures that traffic congestion on the network is avoided. The application layer will still be offering data for transmission to TCP without worrying about the network traffic congestion issues, as the TCP on the sender and receiver sides implement sliding windows of packet buffer. The window size may vary dynamically depending on network traffic.

For the highest possible throughput, it is important that the transmitter is not forced to stop sending by the sliding window protocol earlier than one round-trip delay time (RTT). The limit on the amount of data that it can send before stopping to wait for an acknowledgment should be larger than the bandwidth-delay product of the communications link. If it is not, the protocol will limit the effective bandwidth of the link.

Motivation

In any communication protocol based on automatic repeat request for error control, the receiver must acknowledge received packets. If the transmitter does not receive an acknowledgment within a reasonable time, it re-sends the data.

A transmitter that does not get an acknowledgment cannot know if the receiver actually received the packet; it may be that it was lost or damaged in transmission. If the error detection mechanism reveals corruption, the packet will be ignored by the receiver, and a negative or duplicate acknowledgement will be sent by the receiver. The receiver may also be configured not send any acknowledgement at all. Similarly, the receiver is usually uncertain about whether its acknowledgments are being received. It may be that an acknowledgment was sent, but was lost or corrupted in the transmission medium. In this case, the receiver must acknowledge the retransmission to prevent the data from being continually resent, but must otherwise ignore it.

Protocol operation

The transmitter and receiver each have a current sequence number *nt* and *nr*, respectively. They each also have a window size *wt* and *wr*. The window sizes may vary, but in simpler implementations, they are fixed. The window size must be greater than zero for any progress to be made.

As typically implemented, *nt* is the next packet to be transmitted, i.e., the sequence number of the first packet not yet transmitted. Likewise, *nr* is the first packet not yet received. Both numbers are monotonically increasing with time; they only ever increase.

The receiver may also keep track of the greatest sequence number yet received; the variable *ns* is one more than the greatest sequence number on any accepted packet. This can exceed *ns* by up to $wr-1$, so for simple receivers that only accept packets in order ($wr = 1$), this is the same as *nr*. Note the distinction: all packets before *nr* have been received, no packets after *ns* have been received, and between *nr* and *ns*, some packets have been received. Thus, $nr \le ns \le nt$.

When the receiver receives a packet, it updates its variables appropriately and transmits an acknowledgment with the new *nr*. The transmitter keeps track of the greatest acknowledgment it has received *na*. The transmitter knows that all packets up to, but not including *na* have been received, but is uncertain about packets between *na* and *nt*; i.e. $na \le nr \le nt$.

The sequence numbers always obey the rule that $na \le nr \le ns \le nt \le na + wt$. That is:

- $na \le nr$: Acknowledgements received by the transmitter cannot exceed those sent by the receiver.
- $nr \le ns$: The span of fully received packets cannot extend into the range of never-received packets.
- $ns \le nt$: Packets received cannot exceed packets transmitted.
- $nt \le na + wt$: The greatest packet number sent is limited by the greatest acknowledgement received plus the transmit window size.

Transmitter operation

Whenever the transmitter has data to send, it may transmit up to *wt* packets ahead of the latest acknowledgment *na*. That is, it may transmit packet number *nt* as long as $nt < na+wt$. *nt* is incremented to reflect the transmission.

In the absence of a communication error, the transmitter soon receives an acknowledgment for all the packets it has sent, leaving *na* equal to *nt*. If this does not happen after a reasonable delay, the transmitter must retransmit the packet numbered *na*.

Techniques for defining "reasonable delay" can be extremely elaborate, but they only affect efficiency; the basic reliability of the sliding window protocol does not depend on the details. Similarly, the transmitter may choose to retransmit additional packets between *na* and *nt*, but the decision does not affect the protocol's correctness.

Receiver operation

Every time a packet numbered *x* is received, the receiver checks to see if it falls in the receive window, $nr \le x < nr+wr$. (The simplest receivers have $wr = 1$, and only accept one possibility.) If it falls within the window, the receiver accepts it. If it is numbered *nr*, the receive sequence number is increased by 1, and possibly more if further consecutive packets were previously received and stored. If $x > nr$, the packet is stored until all preceding packets have been received.1(https://en.wikipedia.org/wiki/Sliding_window_protocol#cite_note-1) If $x \ge ns$, the latter is updated to $ns=x+1$.

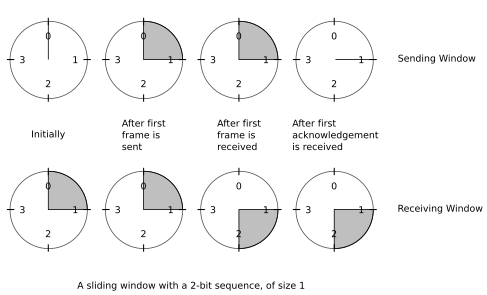
If the packet's number is not within the receive window, the receiver discards it and does not modify *nr* nor *ns*.

Whether the packet was accepted or not, the receiver transmits an acknowledgment containing the current *nr*. (The acknowledgment may also include information about additional packets received between *nr* and *ns*, but that only helps efficiency.)

Note that there is no point having the receive window *wr* larger than the transmit window *wt*, because there is no need to worry about receiving a packet that will never be transmitted; the useful range is $1 \le wr \le wt$.

Sequence number range required

Main article: serial number arithmetic



Sequence numbers modulo 4, with $wr=1$. Initially, $nt=nr=0$

So far, the protocol has been described as if sequence numbers are of unlimited size, ever-increasing. However, rather than transmitting the full sequence number *x* in messages, it is possible to transmit only *x* mod *N*, for some finite *N*. (*N* is usually a power of 2.)

For example, the transmitter will only receive acknowledgments in the range *na* to *nt*, inclusive. Since it guarantees that $nt-na \le wt$, there are at most $wt+1$ possible sequence numbers that could arrive at any given time. Thus, the transmitter can unambiguously decode the acknowledgement sequence number as long as $N > wt$.

A stronger constraint is imposed by the receiver. The operation of the protocol depends on the receiver being able to reliably distinguish new packets (which should be accepted and processed) from retransmissions of old packets (which must be discarded, and the last acknowledgment retransmitted). This can be done given knowledge of the transmitter's window size to infer the transmitter's *na*. After receiving a packet numbered *x*, the receiver knows that $x < nt \le na+wt$, so $na > x-wt$. Thus, packets numbered $x-wt$ or less will never again be retransmitted.

The least sequence number we will ever receive in the future is *ns*−*wt*

The receiver also knows that the transmitter's *na* cannot be greater than the greatest acknowledgment ever sent, which is *nr*. So we will never see a sequence number *na*+*wt* ≤ *nr*+*wt* or greater.

Thus, there are 2*wt* different sequence numbers that the receiver could receive at any one time. It might therefore seem that we must have *N* ≥ 2*wt*. However, the actual limit is lower.

The additional insight is that the receiver does not need to distinguish between sequence numbers that are too low (less than *nr*) or that are too high (greater than or equal to *ns*+*wr*). In either case, the receiver ignores the packet except to retransmit an acknowledgment. Thus, it is only necessary that *N* ≥ *wt*+*wr*. As it is common to have *wr*<*wt* (e.g. see Go-Back-N below), this can permit larger *wt* within a fixed *N*.

Sliding window based part-of-speech tagging is used to part-of-speech tag a text.

A high percentage of words in a natural language are words which out of context can be assigned more than one part of speech. The percentage of these ambiguous words is typically around 30%, although it depends greatly on the language. Solving this problem is very important in many areas of natural language processing. For example in machine translation changing the part-of-speech of a word can dramatically change its translation.

Sliding window based part-of-speech taggers are programs which assign a single part-of-speech to a given lexical form of a word, by looking at a fixed sized "window" of words around the word to be disambiguated.

The two main advantages of this approach are:

- It is possible to automatically train the tagger, getting rid of the need of manually tagging a corpus.
- The tagger can be implemented as a finite state automaton (Mealy machine)

Formal definition

Let

Γ={γ1,γ2,…,γ|Γ|}$\Gamma = \{\gamma_1, \gamma_2, \ldots, \gamma_{|\Gamma|}\}$

be the set of grammatical tags of the application, that is, the set of all possible tags which may be assigned to a word, and let

W={w1,w2,…}$W = \{w1, w2, \ldots\}$

be the vocabulary of the application. Let

T:W→P(Γ)$T : W \to P(\Gamma)$

be a function for morphological analysis which assigns each ww its set of possible tags, T(w)⊆Γ$T(w) \subseteq \Gamma$, that can be implemented by a full-form lexicon, or a morphological analyser. Let

Σ={σ1,σ2,…,σ|Σ|}$\Sigma = \{\sigma_1, \sigma_2, \ldots, \sigma_{|\Sigma|}\}$

be the set of word classes, that in general will be a partition of WW with the restriction that for each σ∈Σ$\sigma \in \Sigma$ all of the words w,Σ,σw, Σ, σ will receive the same set of tags, that is, all of the words in each word class σσ belong to the same ambiguity class.

Normally, ΣΣ is constructed in a way that for high frequency words, each word class contains a single word, while for low frequency words, each word class corresponds to a single ambiguity class. This allows good performance for high frequency ambiguous words, and doesn't require too many parameters for the tagger.

With these definitions it is possible to state problem in the following way: Given a text w[1]w[2]…w[L]∈W*$w[1]w[2]\ldots w[L] \in W^*$ each word w[t]$w[t]$ is assigned a word class T(w[t])∈Σ$T(w[t]) \in \Sigma$ (either by using the lexicon or morphological analyser) in order to get an ambiguously tagged text σ[1]σ[2]…σ[L]∈W*$\sigma[1]\sigma[2]\ldots\sigma[L] \in W^*$. The job of the tagger is to get a tagged text γ[1]γ[2]…γ[L] $\gamma[1]\gamma[2]\ldots\gamma[L]$ (with γ[t]∈T(σ[t])$\gamma[t] \in T(\sigma[t])$) as correct as possible.

A statistical tagger looks for the most probable tag for an ambiguously tagged text σ[1]σ[2]…σ[L]$\sigma[1]\sigma[2]\ldots\sigma[L]$:

γ*[1]…γ*[L]=argmaxγ[t]∈T(σ[t])p(γ[1]…γ[L]σ[1]…σ[L])$\gamma^*[1]\ldots\gamma^*[L] = \arg\max_{\gamma[t]\in T(\sigma[t])} p(\gamma[1]\ldots\gamma[L]\sigma[1]\ldots\sigma[L])$

Using Bayes formula, this is converted into:

γ*[1]…γ*[L]=argmaxγ[t]∈T(σ[t])p(γ[1]…γ[L])p(σ[1]…σ[L]γ[1]…γ[L])$\gamma^*[1]\ldots\gamma^*[L] = \arg\max_{\gamma[t]\in T(\sigma[t])} p(\gamma[1]\ldots\gamma[L])p(\sigma[1]\ldots\sigma[L]\gamma[1]\ldots\gamma[L])$

where p(γ[1]γ[2]…γ[L])$p(\gamma[1]\gamma[2]\ldots\gamma[L])$ is the probability that a particular tag (syntactic probability) and p(σ[1]…σ[L]γ[1]…γ[L])$p(\sigma[1]\ldots\sigma[L]\gamma[1]\ldots\gamma[L])$ is the probability that this tag corresponds to the text σ[1]…σ[L]$\sigma[1]\ldots\sigma[L]$ (lexical probability).

In a Markov model, these probabilities are approximated as products. The syntactic probabilities are modelled by a first order Markov process:

$$p(\gamma[1]\gamma[2]\ldots\gamma[L]) = \prod_{t=1}^{t=L} p(\gamma[t+1]\gamma[t])$$

p(γ[1]γ[2]…γ[L])=∏t=1t=Lp(γ[t+1]γ[t])

where γ[0]$\gamma[0]$ and γ[L+1]$\gamma[L+1]$ are delimiter symbols.

Lexical probabilities are independent of context:

$$p(\sigma[1]\sigma[2]\ldots\sigma[L]\gamma[1]\gamma[2]\ldots\gamma[L]) = \prod_{t=1}^{t=L} p(\sigma[t]\gamma[t])$$

p(σ[1]σ[2]…σ[L]γ[1]γ[2]…γ[L])=∏t=1t=Lp(σ[t]γ[t])

One form of tagging is to approximate the first probability formula:

$$p(\sigma[1]\sigma[2]\ldots\sigma[L]\gamma[1]\gamma[2]\ldots\gamma[L]) = \prod_{t=1}^{t=L} p(\gamma[t]C_{(-)}[t]\sigma[t]C_{(+)}[t])$$

p(σ[1]σ[2]…σ[L]γ[1]γ[2]…γ[L])=∏t=1t=Lp(γ[t]C(−)[t]σ[t]C(+)[t])

where C(−)[t]=σ[t−N(−)]σ[t−N(−)]…σ[t−1]$C_{(-)}[t] = \sigma[t - N_{(-)}]\sigma[t - N_{(-)}]\ldots\sigma[t-1]$ is the right context of the size N(+)$N_{(+)}$.

In this way the sliding window algorithm only has to take into account a context of size N(−)+N(+)+1$N_{(-)} + N_{(+)} + 1$. For most applications N(−)=N(+)=1$N_{(-)} = N_{(+)} = 1$. For example to tag the ambiguous word "run" in the sentence "He runs from danger", only the tags of the words "He" and "from" are needed to be taken into account.

I have actually used the prolog version of wordnet for this as an embbebed synset graph for steiner triples, that i think we can use as a trigram tagger, at the right abstraction level of codepoint partitioning

In computational linguistics, a **trigram tagger** is a statistical method for automatically identifying words as being nouns, verbs, adjectives, adverbs, etc. based on second order Markov models that consider triples of consecutive words. It is trained on a text corpus as a method to predict the next word, taking the product of the probabilities of unigram, bigram and trigram. In speech recognition, algorithms utilizing trigram-tagger score better than those algorithms utilizing IIMM tagger but less well than Net tagger.

The description of the trigram tagger is provided by Brants (2000).

Unified Parallel C (**UPC**) is an extension of the C programming language designed for high-performance computing on large-scale parallel machines, including those with a common global address space (SMP and NUMA) and those with distributed memory (e.g. clusters). The programmer is presented with a single partitioned global address space, where shared variables may be directly read and written by any processor, but each variable is physically associated with a single processor. UPC uses a *single program, multiple data* (SPMD) model of computation in which the amount of parallelism is fixed at program startup time, typically with a single thread of execution per processor.

In order to express parallelism, UPC extends ISO C 99 with the following constructs:

- An explicitly parallel execution model
- A shared address space (`shared` storage qualifier) with thread-local parts (normal variables)
- Synchronization primitives and a memory consistency model
- Explicit communication primitives, e.g. `upc_memput`
- Memory management primitives